

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO TUẦN MỘT
MÔN THỰC TẬP CƠ SỞ

Giảng viên hướng dẫn : Kim Ngọc Bách

Sinh viên thực hiện : Nguyễn Trung Nghĩa

Mã Sinh Viên : B23DCCN602

Hà Nội - 2026

1. Mục tiêu hướng đến

Tuần đầu tiên của dự án tập trung vào việc chuẩn bị các điều kiện cần thiết để có thể triển khai và phát triển hệ thống phát hiện người đội mũ bảo hiểm sử dụng mô hình YOLOv8. Đây là giai đoạn nền tảng nhằm đảm bảo môi trường phát triển ổn định cũng như trang bị các kiến thức lý thuyết cần thiết trước khi tiến hành các bước tiếp theo như xây dựng dataset và huấn luyện mô hình.

- Chuẩn bị môi trường phát triển cho dự án trên máy tính cá nhân.
- Cài đặt các công cụ và thư viện cần thiết phục vụ cho việc huấn luyện mô hình học sâu.
- Kiểm tra khả năng sử dụng GPU thông qua nền tảng CUDA để tăng tốc quá trình huấn luyện.
- Nghiên cứu và củng cố các nền tảng lý thuyết liên quan đến Computer Vision, Deep Learning và Object Detection.
- Tìm hiểu chi tiết hơn về kiến trúc và nguyên lý hoạt động của mô hình YOLOv8.

Việc hoàn thành các mục tiêu trên giúp tạo tiền đề vững chắc cho các giai đoạn tiếp theo của dự án, đặc biệt là giai đoạn xây dựng dataset và huấn luyện mô hình.

2. Chuẩn bị môi trường

Để triển khai các mô hình học sâu trong lĩnh vực thị giác máy tính, việc chuẩn bị môi trường phát triển là bước quan trọng nhằm đảm bảo các công cụ và thư viện hoạt động ổn định và tương thích với nhau. Trong tuần đầu tiên của dự án, môi trường phát triển được thiết lập trên máy tính cá nhân với sự hỗ trợ của GPU NVIDIA nhằm tăng tốc quá trình huấn luyện mô hình.

2.1. Cài đặt Python

Python là ngôn ngữ lập trình được sử dụng rộng rãi trong lĩnh vực trí tuệ nhân tạo và khoa học dữ liệu nhờ cú pháp đơn giản, dễ sử dụng và hệ sinh thái thư viện phong phú. Hầu hết các framework học sâu hiện nay như PyTorch hay TensorFlow đều được phát triển với sự hỗ trợ mạnh mẽ cho Python.

Trong dự án này, Python được lựa chọn làm ngôn ngữ lập trình chính để xây dựng và triển khai hệ thống phát hiện đối tượng. Phiên bản Python 3.11 được sử dụng nhằm đảm bảo khả năng tương thích với các thư viện hiện đại như PyTorch và Ultralytics YOLOv8.

Sau khi cài đặt Python, hệ thống được kiểm tra để đảm bảo Python đã được cài đặt thành công bằng lệnh:

python --version

Nếu hệ thống hiển thị phiên bản Python tương ứng, điều đó cho thấy quá trình cài đặt đã hoàn tất.

2.2. Tạo môi trường ảo (Virtual Environment)

Trong quá trình phát triển phần mềm, đặc biệt là các dự án sử dụng nhiều thư viện khác nhau, việc quản lý môi trường là rất quan trọng. Nếu tất cả các thư viện được cài đặt trực tiếp vào hệ thống chung, có thể xảy ra xung đột giữa các phiên bản thư viện khác nhau.

Để giải quyết vấn đề này, môi trường ảo (Virtual Environment) được sử dụng. Môi trường ảo cho phép tạo ra một không gian làm việc độc lập cho từng dự án, trong đó các thư viện được cài đặt riêng biệt và không ảnh hưởng đến các dự án khác.

Trong dự án này, môi trường ảo được tạo bằng công cụ venv tích hợp sẵn trong Python. Quá trình tạo môi trường ảo được thực hiện bằng lệnh:

py -3.11 -m venv venv

Sau khi tạo môi trường ảo, môi trường được kích hoạt bằng lệnh:

venv\Scripts\activate

Khi môi trường ảo được kích hoạt thành công, hệ thống sẽ hiển thị tên môi trường ở đầu dòng lệnh, cho thấy rằng tất cả các thư viện được cài đặt sau đó sẽ chỉ thuộc về môi trường của dự án.

2.3. Cài đặt thư viện PyTorch

PyTorch là một framework học sâu mã nguồn mở được phát triển bởi Meta AI (trước đây là Facebook AI Research). Đây là một trong những framework phổ biến nhất hiện nay trong lĩnh vực Deep Learning nhờ tính linh hoạt cao và khả năng hỗ trợ GPU mạnh mẽ.

Trong dự án này, PyTorch được sử dụng làm nền tảng để chạy các mô hình học sâu, bao gồm mô hình YOLOv8. Một ưu điểm quan trọng của PyTorch là khả năng tận dụng GPU thông qua nền tảng CUDA nhằm tăng tốc quá trình huấn luyện và suy luận mô hình.

Việc cài đặt PyTorch được thực hiện thông qua lệnh:

`pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121`

Trong đó:

- torch là thư viện chính của PyTorch
- torchvision cung cấp các công cụ xử lý dữ liệu ảnh và các mô hình Computer Vision

Sau khi cài đặt, hệ thống được kiểm tra để đảm bảo PyTorch đã hoạt động đúng.

2.4. Cài đặt mô hình YOLOv8

YOLOv8 là phiên bản mới của dòng mô hình YOLO được phát triển bởi Ultralytics. Đây là một trong những mô hình phát hiện đối tượng hiện đại có khả năng xử lý nhanh và đạt độ chính xác cao trong nhiều bài toán Computer Vision.

Trong dự án này, YOLOv8 được sử dụng làm mô hình chính để phát hiện người đội mũ bảo hiểm trong ảnh và video. Thư viện YOLOv8 được cài đặt thông qua gói Ultralytics bằng lệnh:

`pip install ultralytics`

Sau khi cài đặt, người dùng có thể sử dụng các công cụ có sẵn trong thư viện Ultralytics để huấn luyện và chạy mô hình YOLO.

2.5. Cài đặt các thư viện hỗ trợ

Ngoài các framework học sâu chính, một số thư viện hỗ trợ khác cũng được cài đặt nhằm phục vụ cho quá trình xử lý dữ liệu và hiển thị kết quả.

Các thư viện này bao gồm:

- OpenCV: thư viện xử lý ảnh và video
- Matplotlib: thư viện hỗ trợ trực quan hóa dữ liệu

Quá trình cài đặt được thực hiện thông qua các lệnh:

`pip install opencv-python`

`pip install matplotlib`

Những thư viện này đóng vai trò quan trọng trong việc đọc dữ liệu hình ảnh, xử lý video và hiển thị kết quả phát hiện đối tượng.

2.6. Kiểm tra GPU và CUDA

Đối với các mô hình Deep Learning, việc sử dụng GPU giúp tăng tốc quá trình huấn luyện đáng kể so với việc sử dụng CPU. Trong dự án này, máy tính cá nhân được trang bị GPU NVIDIA hỗ trợ CUDA, cho phép PyTorch tận dụng khả năng tính toán song song của GPU.

Sau khi cài đặt PyTorch và CUDA, việc kiểm tra khả năng sử dụng GPU được thực hiện thông qua đoạn mã Python trong file `gpu_test.py` (chi tiết xem Phụ lục A).

Kết quả cho thấy PyTorch có thể sử dụng GPU để thực hiện các phép tính học sâu. Điều này giúp đảm bảo rằng quá trình huấn luyện mô hình trong các giai đoạn tiếp theo sẽ được thực hiện với hiệu suất cao hơn.

```
(venv) D:\Python\helmet_detection_project>D:/Python/helmet_detection_project/venv/Scripts/python.exe d:/Python/helmet_detection_project/SourceCode/gpu_test.py
KIỂM TRA MÔI TRƯỜNG CUDA
Python version: 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
PyTorch version: 2.5.1+cu121
CUDA available: True
GPU: NVIDIA GeForce RTX 3050 Ti Laptop GPU
CUDA version: 12.1
GPU Memory: 4.29 GB
Phép tính trên GPU hoạt động
OpenCV version: 4.13.0
Ultralytics YOLO import thành công
KIỂM TRA HOÀN TẤT
```

3. Nghiên cứu cơ sở lý thuyết

Bên cạnh việc chuẩn bị môi trường phát triển, trong tuần đầu tiên của dự án, em cũng tiến hành nghiên cứu các kiến thức lý thuyết nền tảng liên quan đến bài toán phát hiện đối tượng trong lĩnh vực thị giác máy tính. Việc nắm vững các cơ sở lý thuyết giúp hiểu rõ nguyên lý hoạt động của các mô hình học sâu, từ đó có thể lựa chọn phương pháp phù hợp và tối ưu hóa hệ thống trong quá trình triển khai.

Các nội dung lý thuyết được nghiên cứu bao gồm Computer Vision, Object Detection, Deep Learning, Convolutional Neural Network và mô hình YOLO.

3.1. Computer Vision

Computer Vision là một lĩnh vực quan trọng trong trí tuệ nhân tạo, tập trung vào việc giúp máy tính có khả năng hiểu và phân tích thông tin từ hình ảnh hoặc video tương

tự như cách con người quan sát thế giới. Mục tiêu của Computer Vision là cho phép máy tính tự động nhận diện, phân tích và đưa ra quyết định dựa trên dữ liệu hình ảnh.

Trong thực tế, Computer Vision được ứng dụng rộng rãi trong nhiều lĩnh vực khác nhau như:

- Hệ thống giám sát an ninh
- Xe tự lái
- Nhận dạng khuôn mặt
- Phân tích hình ảnh y tế
- Hệ thống giao thông thông minh

Một hệ thống Computer Vision thường bao gồm các bước cơ bản sau:

- Thu thập dữ liệu hình ảnh hoặc video
- Tiền xử lý dữ liệu (preprocessing)
- Trích xuất đặc trưng (feature extraction)
- Phân tích và nhận dạng đối tượng
- Đưa ra kết quả hoặc quyết định

Trong những năm gần đây, sự phát triển mạnh mẽ của Deep Learning đã giúp các hệ thống Computer Vision đạt được độ chính xác cao hơn đáng kể so với các phương pháp truyền thống.

3.2. Bài toán Object Detection

Object Detection là một trong những bài toán quan trọng trong lĩnh vực Computer Vision. Mục tiêu của bài toán này là xác định vị trí và loại của các đối tượng xuất hiện trong một hình ảnh hoặc video.

Khác với bài toán Image Classification, trong đó mô hình chỉ cần xác định nội dung chính của toàn bộ bức ảnh, Object Detection yêu cầu mô hình phải thực hiện hai nhiệm vụ đồng thời:

- Xác định vị trí của đối tượng trong ảnh
- Phân loại đối tượng đó thuộc lớp nào

Vị trí của đối tượng thường được biểu diễn thông qua bounding box, là một hình chữ nhật bao quanh đối tượng cần phát hiện. Bounding box thường được biểu diễn bằng bốn tham số:

- Tọa độ x của tâm

- Tọa độ y của tâm
- Chiều rộng của bounding box
- Chiều cao của bounding box

Kết quả của một mô hình Object Detection thường bao gồm:

- Bounding box của đối tượng
- Nhãn lớp (class label)
- Độ tin cậy của dự đoán (confidence score)

Hiện nay có hai hướng tiếp cận chính trong bài toán Object Detection:

a) Two-stage detectors

Các mô hình two-stage detector thực hiện quá trình phát hiện đối tượng qua hai bước:

- Đề xuất vùng có khả năng chứa đối tượng (region proposal)
- Phân loại đối tượng trong các vùng đó

Các mô hình tiêu biểu của phương pháp này bao gồm:

- R-CNN
- Fast R-CNN
- Faster R-CNN

Ưu điểm của phương pháp này là độ chính xác cao, tuy nhiên tốc độ xử lý thường chậm hơn.

b) One-stage detectors

Khác với phương pháp two-stage, các mô hình one-stage detector thực hiện việc phát hiện và phân loại đối tượng chỉ trong một bước duy nhất. Điều này giúp tăng đáng kể tốc độ xử lý.

Một số mô hình tiêu biểu của phương pháp này bao gồm:

- YOLO (You Only Look Once)
- SSD (Single Shot Detector)

Trong dự án này, mô hình YOLOv8 được lựa chọn vì khả năng phát hiện đối tượng nhanh và phù hợp với các ứng dụng thời gian thực.

3.3. Deep Learning

Deep Learning là một nhánh của Machine Learning sử dụng các mạng nơ-ron nhân tạo nhiều tầng để học các biểu diễn dữ liệu phức tạp. Khác với các phương pháp học

máy truyền thống, Deep Learning có khả năng tự động học các đặc trưng từ dữ liệu mà không cần thiết kế thủ công.

Một mô hình Deep Learning thường bao gồm nhiều lớp (layers), trong đó mỗi lớp thực hiện một phép biến đổi đối với dữ liệu đầu vào. Các lớp này được kết nối với nhau để tạo thành một mạng nơ-ron sâu.

Quá trình huấn luyện mô hình Deep Learning thường bao gồm các bước sau:

1. Forward propagation

Dữ liệu đầu vào được truyền qua các lớp của mạng để tạo ra kết quả dự đoán.

2. Tính toán hàm mất mát (loss function)

So sánh kết quả dự đoán với nhãn thực tế để xác định sai số của mô hình.

3. Backpropagation

Sai số được lan truyền ngược lại qua mạng để điều chỉnh các trọng số.

4. Cập nhật trọng số (weight update)

Các trọng số của mạng được cập nhật nhằm giảm giá trị của hàm mất mát.

Nhờ khả năng học biểu diễn dữ liệu phức tạp, Deep Learning đã đạt được nhiều thành tựu nổi bật trong các lĩnh vực như nhận dạng hình ảnh, xử lý ngôn ngữ tự nhiên và nhận dạng giọng nói.

3.4. Convolutional Neural Network (CNN)

Convolutional Neural Network (CNN) là một loại mạng nơ-ron được thiết kế đặc biệt để xử lý dữ liệu dạng lưới, chẳng hạn như hình ảnh. CNN là nền tảng của hầu hết các mô hình Computer Vision hiện đại.

Một mạng CNN thường bao gồm các loại lớp chính sau:

a) Convolution Layer

Lớp convolution thực hiện phép tích chập giữa ảnh đầu vào và các bộ lọc (filters) để trích xuất các đặc trưng của ảnh. Các bộ lọc này có thể phát hiện các đặc điểm như cạnh, góc hoặc hoa văn trong ảnh.

b) Activation Function

Sau khi thực hiện phép convolution, hàm kích hoạt được sử dụng để đưa tính phi tuyến vào mạng. Một trong những hàm kích hoạt phổ biến nhất là ReLU (Rectified Linear Unit).

c) Pooling Layer

Pooling layer được sử dụng để giảm kích thước của feature map, giúp giảm số lượng tham số và tăng hiệu quả tính toán. Hai phương pháp pooling phổ biến là:

- Max Pooling
- Average Pooling

d) Fully Connected Layer

Ở các lớp cuối của mạng CNN, các lớp fully connected được sử dụng để thực hiện việc phân loại đối tượng dựa trên các đặc trưng đã được trích xuất.

CNN có khả năng tự động học các đặc trưng từ dữ liệu hình ảnh, giúp giảm đáng kể sự phụ thuộc vào việc thiết kế đặc trưng thủ công.

3.5. Mô hình YOLO

YOLO (You Only Look Once) là một trong những mô hình phát hiện đối tượng phổ biến nhất trong lĩnh vực Computer Vision. Mô hình này được giới thiệu lần đầu vào năm 2016 và nhanh chóng trở thành một trong những phương pháp hiệu quả cho bài toán Object Detection.

Ý tưởng chính của YOLO là chia ảnh đầu vào thành một lưới (grid). Mỗi ô trong lưới có nhiệm vụ dự đoán các bounding box và xác suất đối tượng xuất hiện trong khu vực đó.

So với các phương pháp truyền thống, YOLO có một số ưu điểm quan trọng:

- Tốc độ xử lý rất nhanh
- Có thể hoạt động gần thời gian thực
- Kiến trúc mạng đơn giản hơn so với nhiều phương pháp khác

Trong quá trình phát triển, YOLO đã trải qua nhiều phiên bản cải tiến như YOLOv3, YOLOv5, YOLOv8, YOLOv10 và mới nhất là YOLO26.

3.6. Mô hình YOLOv8

YOLOv8 là phiên bản cải tiến của dòng mô hình YOLO do Ultralytics phát triển. Phiên bản này được thiết kế nhằm cải thiện cả độ chính xác và hiệu suất xử lý so với các phiên bản trước.

YOLOv8 sử dụng kiến trúc mạng hiện đại với các thành phần chính:

- Backbone: trích xuất đặc trưng từ ảnh đầu vào
- Neck: kết hợp các đặc trưng ở nhiều mức khác nhau
- Head: dự đoán bounding box và nhãn đối tượng

Quá trình phát hiện đối tượng của YOLOv8 có thể được mô tả qua các bước:

1. Ảnh đầu vào được đưa vào mạng CNN để trích xuất đặc trưng.
2. Các đặc trưng được xử lý qua nhiều tầng mạng để phát hiện các vùng có khả năng chứa đối tượng.
3. Mô hình dự đoán vị trí bounding box và xác suất của từng lớp đối tượng.
4. Thuật toán Non-Maximum Suppression (NMS) được áp dụng để loại bỏ các bounding box trùng lặp.

YOLOv8 cung cấp nhiều phiên bản mô hình khác nhau như:

- YOLOv8n (nano)
- YOLOv8s (small)
- YOLOv8m (medium)
- YOLOv8l (large)
- YOLOv8x (extra large)

Trong dự án này, các phiên bản nhẹ như YOLOv8n hoặc YOLOv8s được ưu tiên sử dụng nhằm đảm bảo tốc độ xử lý nhanh và phù hợp với cấu hình phần cứng hiện có.

3.7. Các chỉ số đánh giá mô hình phát hiện đối tượng

Trong các bài toán Object Detection, việc đánh giá hiệu suất của mô hình không chỉ dựa trên độ chính xác phân loại mà còn phải xem xét khả năng xác định vị trí của đối tượng trong ảnh. Do đó, nhiều chỉ số khác nhau được sử dụng để đánh giá mô hình một cách toàn diện, bao gồm Intersection over Union (IoU), Precision, Recall và mean Average Precision (mAP).

3.7.1. Intersection over Union (IoU)

Intersection over Union (IoU) là một chỉ số dùng để đo mức độ trùng khớp giữa bounding box dự đoán của mô hình và bounding box thực tế trong dữ liệu gán nhãn. IoU được tính bằng tỷ lệ giữa diện tích phần giao nhau và diện tích phần hợp của hai bounding box.

Công thức của IoU:

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

Trong đó:

- Area of Overlap: diện tích phần giao giữa hai bounding box
- Area of Union: tổng diện tích của hai bounding box sau khi loại bỏ phần giao

Giá trị IoU nằm trong khoảng từ 0 đến 1:

- IoU = 0: hai bounding box không giao nhau
- IoU = 1: hai bounding box trùng khớp hoàn toàn

Trong các hệ thống phát hiện đối tượng, một dự đoán thường được xem là đúng (True Positive) nếu giá trị IoU lớn hơn một ngưỡng nhất định, ví dụ 0.5.

3.7.2. Precision

Precision là chỉ số đo lường tỷ lệ dự đoán đúng trong tổng số các dự đoán của mô hình.

Công thức:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Trong đó:

- TP (True Positive): số đối tượng được phát hiện đúng
- FP (False Positive): số đối tượng được phát hiện nhưng thực tế không tồn tại

Precision cao cho thấy mô hình ít tạo ra các dự đoán sai.

3.7.3. Recall

Recall là chỉ số đo lường khả năng phát hiện đầy đủ các đối tượng thực sự tồn tại trong ảnh.

Công thức:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Trong đó:

- FN (False Negative): số đối tượng thực tế tồn tại nhưng mô hình không phát hiện được

Recall cao cho thấy mô hình không bỏ sót nhiều đối tượng.

3.7.4. Average Precision (AP)

Average Precision (AP) là chỉ số được sử dụng để đánh giá hiệu suất phát hiện của mô hình đối với một lớp đối tượng cụ thể.

AP được tính bằng cách tích hợp diện tích dưới đường cong Precision-Recall.

Quá trình tính AP bao gồm các bước:

1. Sắp xếp các dự đoán theo độ tin cậy (confidence score)
2. Tính Precision và Recall tại các ngưỡng khác nhau
3. Vẽ đường cong Precision-Recall
4. Tính diện tích dưới đường cong

Giá trị AP càng cao thì mô hình càng có khả năng phát hiện chính xác đối tượng.

3.7.5. mean Average Precision (mAP)

mean Average Precision (mAP) là chỉ số tổng hợp được sử dụng phổ biến trong các bài toán Object Detection để đánh giá hiệu suất chung của mô hình.

mAP được tính bằng trung bình của các giá trị Average Precision trên tất cả các lớp đối tượng.

Ví dụ:

$$\text{mAP} = \frac{1}{N} \sum AP_i$$

Trong đó:

- N là số lớp đối tượng
- AP_i là Average Precision của lớp thứ i

Trong nhiều benchmark phổ biến, mAP thường được tính tại các ngưỡng IoU khác nhau, ví dụ:

- $\text{mAP}@0.5$: $\text{IoU} \geq 0.5$
- $\text{mAP}@0.5:0.95$: trung bình mAP tại các ngưỡng IoU từ 0.5 đến 0.95

Đối với mô hình YOLOv8, hai chỉ số này thường được sử dụng để đánh giá hiệu suất của mô hình sau khi huấn luyện.

4. Kết quả đạt được trong tuần 1

Sau tuần đầu tiên thực hiện dự án, các kết quả chính đạt được bao gồm:

- Xác định rõ hướng nghiên cứu và phạm vi của đề tài.
- Thiết lập thành công môi trường phát triển cho dự án trên máy tính cá nhân.
- Cài đặt và kiểm tra hoạt động của các thư viện cần thiết như PyTorch và Ultralytics YOLOv8.
- Xác nhận GPU NVIDIA hoạt động bình thường thông qua CUDA.
- Nghiên cứu và củng cố các kiến thức lý thuyết liên quan đến Computer Vision, Deep Learning và Object Detection.
- Tìm hiểu nguyên lý hoạt động của mô hình YOLOv8.

Những kết quả này tạo nền tảng vững chắc cho các giai đoạn tiếp theo của dự án, đặc biệt là quá trình xây dựng dataset và huấn luyện mô hình.

5. Khó khăn gặp phải và cách khắc phục

Trong quá trình thực hiện tuần đầu tiên của dự án, việc thực hiện đã gặp một số khó khăn liên quan đến việc thiết lập môi trường phát triển và lựa chọn phiên bản phần mềm phù hợp. Tuy nhiên, các vấn đề này đã được nghiên cứu và giải quyết nhằm đảm bảo hệ thống có thể hoạt động ổn định.

5.1. Khó khăn về tương thích phiên bản phần mềm

Một trong những khó khăn đầu tiên là vấn đề tương thích giữa các phiên bản Python, PyTorch và CUDA. Một số phiên bản Python mới có thể chưa được hỗ trợ đầy đủ bởi các thư viện học sâu, dẫn đến việc cài đặt thư viện gặp lỗi hoặc không thể sử dụng GPU.

Để khắc phục vấn đề này, em đã tiến hành tìm hiểu tài liệu chính thức của các thư viện và lựa chọn phiên bản Python 3.11 cùng với PyTorch hỗ trợ CUDA 12.1 nhằm đảm bảo khả năng tương thích và tận dụng được GPU của hệ thống.

5.2. Khó khăn trong việc cài đặt PyTorch với CUDA

Trong quá trình cài đặt PyTorch, ban đầu hệ thống chỉ nhận phiên bản CPU và không sử dụng được GPU. Điều này dẫn đến việc không thể tận dụng khả năng tính toán song song của GPU trong quá trình huấn luyện mô hình.

Để giải quyết vấn đề này, em đã tiến hành:

- Kiểm tra phiên bản CUDA được hỗ trợ bởi PyTorch
- Sử dụng đúng repository của PyTorch để cài đặt phiên bản hỗ trợ CUDA
- Kiểm tra lại môi trường Python và virtual environment

Sau khi thực hiện các bước trên, hệ thống đã nhận diện thành công GPU NVIDIA RTX 3050 Ti Laptop và PyTorch có thể sử dụng CUDA để tăng tốc tính toán.

5.3. Khó khăn trong việc lựa chọn công cụ phát triển

Một vấn đề khác trong quá trình chuẩn bị dự án là việc lựa chọn môi trường huấn luyện phù hợp. Ban đầu có hai lựa chọn chính:

- Sử dụng môi trường local trên máy cá nhân
- Sử dụng nền tảng điện toán đám mây như Google Colab

Sau khi phân tích cấu hình phần cứng của máy cá nhân và yêu cầu của dự án, em quyết định ưu tiên sử dụng môi trường local để thuận tiện cho việc phát triển, debug và quản lý dữ liệu. Google Colab được xem là phương án dự phòng trong trường hợp cần mở rộng tài nguyên tính toán.

6. Kế hoạch thực hiện trong tuần tiếp theo

Sau khi hoàn thành việc chuẩn bị môi trường và nghiên cứu cơ sở lý thuyết trong tuần đầu tiên, các công việc chính trong tuần tiếp theo sẽ tập trung vào việc xây dựng dataset phục vụ cho quá trình huấn luyện mô hình.

Các nhiệm vụ cụ thể bao gồm:

6.1. Thu thập dữ liệu

Em sẽ tiến hành thu thập các hình ảnh và video liên quan đến người tham gia giao thông bằng xe máy, đặc biệt tập trung vào hai trạng thái:

- Người đội mũ bảo hiểm
- Người không đội mũ bảo hiểm

Nguồn dữ liệu có thể bao gồm:

- Dataset công khai trên Internet
- Hình ảnh từ các bộ dữ liệu về giao thông
- Video hoặc hình ảnh từ các nguồn mở

6.2. Gán nhãn dữ liệu (Data Annotation)

Sau khi thu thập dữ liệu, bước tiếp theo là gán nhãn cho các đối tượng trong ảnh. Quá trình gán nhãn bao gồm việc xác định vị trí của đối tượng và phân loại trạng thái của đối tượng.

Các công cụ gán nhãn dữ liệu phổ biến có thể được sử dụng bao gồm:

- LabelImg
- Roboflow
- CVAT

Trong dự án này, em sẽ lựa chọn công cụ phù hợp để tạo các bounding box bao quanh đối tượng và lưu trữ dữ liệu theo định dạng YOLO.

6.3. Chuẩn hóa dữ liệu theo định dạng YOLO

Để có thể sử dụng dataset cho quá trình huấn luyện mô hình YOLOv8, dữ liệu cần được chuyển đổi sang định dạng phù hợp.

Mỗi ảnh trong dataset sẽ có một file nhãn tương ứng, trong đó chứa các thông tin:

- Class id của đối tượng
- Tọa độ tâm của bounding box
- Chiều rộng của bounding box
- Chiều cao của bounding box

Các giá trị này được chuẩn hóa theo kích thước của ảnh để phù hợp với yêu cầu của mô hình YOLO.

6.4. Tổ chức cấu trúc dataset

Dataset sẽ được chia thành ba tập dữ liệu chính:

- Training set: dùng để huấn luyện mô hình
- Validation set: dùng để đánh giá mô hình trong quá trình huấn luyện
- Test set: dùng để đánh giá hiệu suất cuối cùng của mô hình

Việc tổ chức dataset hợp lý sẽ giúp quá trình huấn luyện đạt hiệu quả cao hơn và hạn chế hiện tượng overfitting.

6.5. Chuẩn bị cho quá trình huấn luyện mô hình

Sau khi dataset được hoàn thiện, em sẽ tiến hành:

- Tạo file cấu hình dataset cho YOLO
- Kiểm tra dữ liệu trước khi huấn luyện
- Chuẩn bị các tham số huấn luyện cơ bản

Những bước này sẽ tạo nền tảng cho quá trình huấn luyện mô hình YOLOv8 trong các tuần tiếp theo.

TÀI LIỆU THAM KHẢO

[1] Ultralytics. (2023). “YOLOv8 documentation.” 2023. [Online]. Từ:

<https://docs.ultralytics.com/models/yolov8/> [Truy cập ngày 06/03/2026].

[2] J. Redmon, S. Divvala, R. Girshick, & A. Farhadi. “You Only Look Once: Unified, Real-Time Object Detection”, trong Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), trang 779 - 788, 2016.

[3] I. Goodfellow, Y. Bengio, & A. Courville. Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

[4] LeCun, Y., Bengio, Y., & Hinton, G.. “Deep learning,” Nature, tập 521, số 7553, trang 436-444, 2015.

[5] NVIDIA Corporation. (2024). “CUDA Toolkit Documentation.” 2024. [Online]. Từ: <https://docs.nvidia.com/cuda/> [Truy cập ngày 06/03/2026]

[6] Paszke, A., Gross, S., Massa, F., và cộng sự. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”, trong Advances in Neural Information Processing Systems (NeurIPS), tập 32, 2019.

PHỤ LỤC

Phụ lục A: file gpu_test.py

```
import torch
import cv2
import sys

print("KIỂM TRA MÔI TRƯỜNG CUDA")

# Kiểm tra Python
print(f"Python version: {sys.version}")

# Kiểm tra PyTorch và CUDA
print(f"PyTorch version: {torch.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")

if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"CUDA version: {torch.version.cuda}")
    print(f"GPU Memory: {torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")

    # Test tensor trên GPU
    x = torch.randn(1000, 1000).cuda()
    y = torch.randn(1000, 1000).cuda()
    z = x @ y
    print("Phép tính trên GPU hoạt động")
else:
    print("CUDA không khả dụng")

# Kiểm tra OpenCV
print(f"OpenCV version: {cv2.__version__}")

# Kiểm tra ultralytics
try:
    from ultralytics import YOLO
    print("Ultralytics YOLO import thành công")
except ImportError as e:
    print(f"Lỗi import ultralytics: {e}")

print("KIỂM TRA HOÀN TẤT")
```

